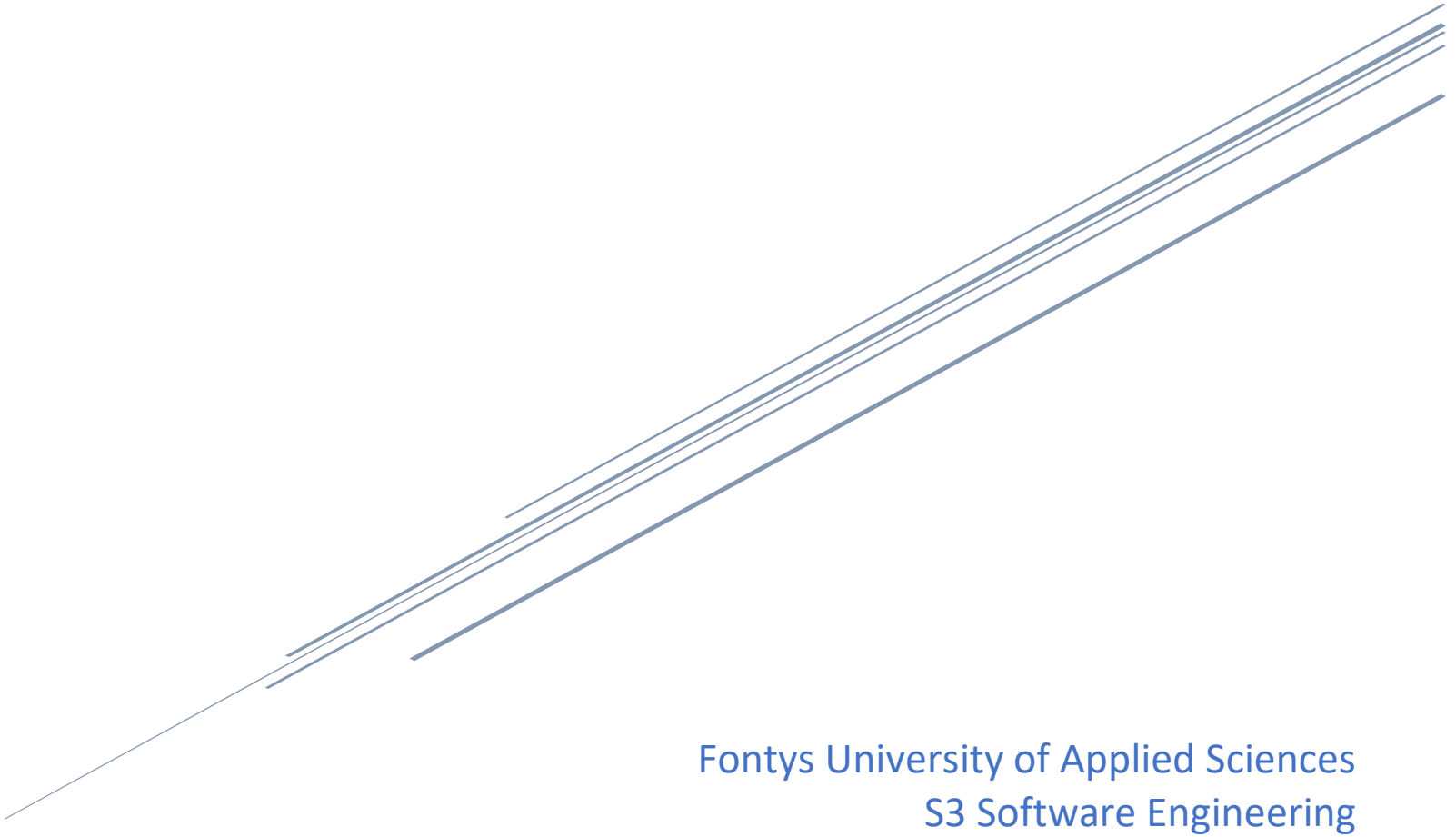
Security Report

By Beatrice Marro



Fontys University of Applied Sciences
S3 Software Engineering

Contents

Introduction

OWASP Security Risks is a resource defining the potential security risks of a web application. This document is valuable to developers when designing their application, as to ensure that it is safe from any malicious activities which may lead to severe consequences, such as providing access of restricted content of the web app to an unintended user.

It is important for developers to consider these top 10 security risks to not only design, but also test their application, ensuring that it is safe from the most common risks a web application is prone to.

OWASP Top 10 Security Risks

Risk	Likelihood	Impact	Risk	Actions possible	Planned
A01: Broken Access Control	High	Severe	Medium	1. Add security which bans the tampering of a JWT token to modify account ids, roles, and creation time. 2. Implement business limit requirements limiting some actions, for instance after attempting to login with invalid credentials after 5 attempts, an account should be frozen for a limited amount of time. Log access control failures to administrators to identify suspicious activities. 3. Limit API access to a limited number of requests in a specific amount of time to prevent the user of automated attack tooling. 4. Follow OAuth standards to revoke access for stateless JWT token.	Yes
A02: Cryptographic Failures	High	Severe	High	1. Do not store sensitive data which is unnecessary to the application – review all data stored for the various account types and determine which of these may be unnecessary for the functionalities of the application. 2. Sensitive data, which is unused should be deleted, as to not unnecessarily store vulnerable data. 3. Store and transmit data securely in an encrypted state over a secure protocol such as TLC, over HTTPS. Encryption should consist of a level of cryptographic complexity and randomness which should not be easy to predict or understand. 4. Allow users for an option to delete all their data from their application – delete their account and all data related to it permanently.	No

A03: Injection	Medium	Less Severe	Low	1. Add more input validation.	Yes
A04: Insecure Design	Relatively Low	Severe	Relatively High	1. Implement SOLID where this is relevant to securing the design of the application, such as enforcing the inheritance which is currently implemented.	Yes
A05: Security Misconfiguration	Low	Severe	Relatively Low	1. Inclusion of security measures in user stories. 2. Inclusion of more misuse (security) unit tests.	No
A06: Vulnerable and Outdated Components	Low	Less Severe	Relatively Low	1. Delete unused/unnecessary features, files, and components from the application. 2. Delete unused dependencies. 3. Make a document of all dependencies and the versions of each item used within the application, regularly check on these dependencies for security risks and check against these.	No
A07: Identification and Authentication Failures	High	Severe	High	1. Use of multi-factor authentication. 2. Various checks performed on a password to ensure that it meets certain security standards. 3. Encouraging of password rotation (changing passwords after some time) 4. Log and save login attempts, when a suspicious amount of incorrect login attempts is performed, the admin should be alerted.	Yes (partly)
A08: Software and Data Integrity Failures	Low	Much Less Severe	Low	1. Use digital signatures to ensure that when the software is downloaded, it is done from the legitimate source. 2. Use security tools to ensure that components do not consist of security vulnerabilities. 3. Implement a code review process to check the code before it is run in the pipeline.	No
A09: Security Logging and Monitoring Failures	High	Severe	High	1. Log events such as login activities, with sufficient information, particularly when suspicious activity arises, such as several unsuccessful logins beyond the acceptable amount. 2. Check logs on a regular basis for suspicious activity. 3. Implement regular penetration testing.	No
A10: Server-Side Request Forgery (SSRF)	Low	Less Severe	Very Low	1. Improve URL consistency to avoid TOCTOU race conditions, for example checking that any links provided in the app are in fact valid and originate from within the REST API itself.	No

Reasoning

A01: Broken Access Control

Definition:

Broken Access Control is a security threat that exists when users can gain access to resources that they should not be able to gain access to.

Implementations:

1. Denial of all resources other than publicly accessible resources, such as approved articles.
2. Limitation of access within application through CORS configuration.

Current Security Risks:

1. Application should limit the amount of business requests that can be made, for instance limiting the number of requests accepted by the API in a specific timeframe.
2. Log access control failures should display any continuous unsuccessful attempts related to logging into an account, as this is an identifier of potentially malicious activity.
3. Saving data such as last login in the database, this can be useful when identifying security issues by knowing the time during which the last login occurred by a particular user.
4. Reduce the lifetime of a JWT token to minimize the amount of time available for a hacker to perform malicious activities. Alternatively, OAuth standards may be followed in use cases when a JWT token consisting of longer lifetime is required.

Impact:

Broken Access Control can have a **medium** impact on my app as users should not be able to gain information that they should not have access to in the first place, such as unpublished articles, or something riskier, such as being able to access and login to the account of an admin or journalist, and having access to the publishing and approval of articles.

A02: Cryptographic Failures

Definition:

Cryptographic Failures refers to when the data of an application is not stored and/or transmitted securely, making its data vulnerable to cyber-attacks.

Implementations:

1. Passwords are hashed and salted using BCrypt.
2. Use of a unique JWT secret token.

Current Security Risks:

1. Storing of many sensitive data for an indefinite duration, in a real scenario, after storing sensitive data beyond a certain period it should be automatically deleted.
2. Data is not stored nor transmitted in an encrypted state, which is not secure as eavesdroppers could intercept and retrieve the contents of this data or access the data within the database, and instantly be able to understand its contents, including sensitive data.
3. Data is sent and received over HTTP, which is not secure, unlike HTTPS as the data is not sent in an encrypted state, making the data vulnerable to being understood as it is being sent.

Impact:

Cryptographic Failure can have a **high** impact on my app, as the data is not transmitted nor stored in an encrypted state (except for the passwords), this means that all the data of the application can potentially be intercepted or accessed and read. Due to this, lots of sensitive information, such as user's account information and unpublished article information may be sold or made public, which itself is very severe, and can cause several issues arising from this, such as using credentials of another user and impersonating them.

A03: Injection

Definition:

Injection refers to being able to inject data into queries to be able to gain data which should not be accessed.

Implementations:

1. Use of JPA opposed to using SQL queries or use of JPQL when it is not needed (used as much as possible). This leads to the insertion of sanitized fields within the queries.
2. Input validation – This helps towards validating the data input by a user before sending it to the repository and executing this within a query.

Current Security Risks:

The security risks related to the use of plain JPA are very low in terms of SQL injection. SQL injection is primarily a risk where native SQL queries are used, or alternatively where JPQL queries are used (but less of a risk than when native SQL is used).

Impact:

The potential impact of SQL injection on my application is **low** this is due to the risk of this occurring being low. If the risk were higher, the impact itself would certainly increase, as this could enable users to gain data that they should not have access to through gaining access to undesired actions with the database, which could go to a very harmful extend, such as the deletion of the entire database, or theft of user data.

A04: Insecure Design

Definition:

An insecure design means having a design which does not adhere to security standards and principles, therefore making it prone to potential attacks.

Implementations:

1. Use of libraries of components to make the application secure – Use of authentication and authorization implementations in backend.
2. Implementation of plausibility checks at various tiers of the application – Implementation of validation is frontend, backend and JPA entities to ensure that the data passed between different tiers of the application is valid and can safely be executed or inserted into the database.
3. Inclusion of unit tests consisting of misuse cases, such as creating an article with an invalid user type set as the author of the article (where the account type should be journalist).

Current Security Risks:

1. Inclusion of security risks in the user stories, for example, in a user story about updating password, include security implementations relevant to this, like multi-factor authentication.
2. Unit tests contain some misuse cases, but no misuse case related to for instance, logging in to an account unsuccessfully multiple time, and an exception being thrown for security reasons.

Impact:

Having a potentially insecure design can have a **relatively high** impact on my app as its functionalities could be exploited if there is a security risk present, I think that my application is quite secure in terms of having a good design, therefore while the impact of this is occurring is high, as it could have a very negative impact on my app, the likelihood this happening is relatively low compared to some other security risks, such as Cryptographic failure.

A05: Security Misconfiguration

Definition:

Security Misconfiguration is when application has incorrect or insufficiently secure security configurations, leaving the data of the application at risk from potential attacks.

Implementations:

1. Debug mode is turned off, so no unnecessary stack-traces or information is logged, thus preventing this from being used to find vulnerabilities in the application.

Currently Security Risks:

1. Running my database in a production environment with its current configurations, given that with the current properties file it is not protected enough, this may be prone to cyber-attacks given that malicious SQL code could be injected into the database, granting the user with the access rights to create new users, and thus gain improper access of the system via this account creation.

Impact:

The impact of security misconfiguration mode is currently **relatively low** for my application as much of this security risk relates to the use of a server, which in my case is not used. This security risk from my current implementation is something that I should implement continuously, such as keeping the components, libraries, and frameworks up to date as to avoid any security risks arising in the older versions.

A06: Vulnerable and Outdated Components

Definition:

The use of vulnerable and outdated components puts an application at risk as it makes use of components which are no longer supported by their developers, hence making the components used within the application at risk.

Implementations:

1. Deleting unused dependencies when found - When I find any unused dependencies in the app, I remove them.

Current Security Risks:

1. Use of up-to-date components, libraries, and frameworks.
2. No documentation for the dependencies used with the backend and frontend applications.
3. Unused dependencies are likely to still exit in the app.
4. Delete an unused features or components, as for instance, unused components may be used as a back door to gain access to the application.
5. Use of Docker with only one account, this means that I have access to all administrator actions regarding my Docker containers, meaning that if I wanted to, I could delete the whole database container, as I have access to it. In a real-world scenario there would be levels of access implemented it terms of Docker accounts, where different employees would utilize different accounts, each with different actions that can be executed, in this was only an administrator can carry out actions such as deleting an entire database, and a lower-level employee cannot do this, which prevents data-loss, both accidental and intentional by employees which should not have access to certain privileges.

Impact:

The impact of having vulnerable and outdated components is **relatively low** in my app, as I think that I do not have many unused components and dependencies present in my app which could lead to threats such as back doors.

A07: Identification and Authentication Failures

Definition:

Identification and Authentication Failures can occur when a system is unable to determine the identity and role of a user within the system, this also refers to the security of user information related to Authentication procedures, such as the use of secure passwords.

Implementations:

1. No hard-coded accounts are stored or created in the application.
2. Some checks (minor) are performed to ensure that the length of a password meets a certain standard, should be greater than 8 characters for improved security.

Current Security Risks:

1. No use of multi-factor authentication, making it possible to for instance, steal and use the account of another person to login, through strategies such as phishing.
2. When a password is created or changed, no checks are performed to check against the worst 10,000 passwords list.
3. Password length is checked, but password complexity and rotation policies are not implemented (with NIST guidelines).
4. No logs of login attempts.
5. No alerts displayed to administrators when suspicious password activities are carried out, for example a suspicious amount of unsuccessful login attempts from one user.
6. Data relative to login activities, such as last login not saved in database.

Impact:

The impact of identification and authentication failures in my app is **high** as having insecure authorization can make it easy for account data to be hacked and used to impersonate a user and potentially accessing sensitive information, such as accessing unpublished articles or admin actions related to publishing articles.

A08: Software and Data Integrity Failures

Definition:

Software and Data Integrity failures may happen when the code lacks the necessary protection, for both the application code and the security measures for its infrastructure.

Current Security Risks:

1. Libraries or dependencies may be consuming faulty repositories.
2. Test components using software supply chain security tools to check their vulnerabilities.
3. No review processes implemented before the code is run in the pipeline.

Impact:

The impact of Software and Data integrity failures is **low** in my app as this is much less likely to occur than other security risks, such as Cryptographic Failure, and is aimed at some topics which are not relevant to my app. Despite this, Software and Data integrity failures could have a very large impact on a real-world app which is deployed.

A09: Security Logging and Monitoring Failures

Definition:

Security Logging and Monitoring failures refers to practices which involve the logging of system actions and failures to be able to identify and suspicious activity.

Implementations:

1. Errors generate clear log messages

Current Security Risks:

1. No logs currently implemented for events such as login, or any otherwise potentially suspicious activities arising in the system
2. No penetration testing done

Impact:

Security logging and monitoring failures has a **high** impact in my app, given that if any security issues were to arise related to account credentials, then retrieving the relevant information related to any malicious activities is very important to be able to retrace any suspicious activities. Also, currently no penetration tests have been done to simulate a hacker trying to gain access to the system, so there are likely to be various security vulnerabilities.

A10: Server-Side Request Forgery (SSRF)

Definition:

Server-Side Request Forgery is when a server is exploited by making malicious requests to gain access and alter its resources.

Implementations:

1. Client data is sanitized and validated (validation occurs both on the client and server-side of the application).
2. No raw responses displayed to users (minimizes users from viewing confidential information relative to the app)

Current Security Risks:

1. URLs provided for links used within the app are not checked that they originate from the Rest API itself, which may make them prone to TOCTOU (time of check, time of use) – These are attacks which consist of first checking the existence of a file, and then reading the contents of this shared file and processes its data, this could for instance occur to a sensitive configuration file.
2. race conditions.

(The Network risks are not included in the implementations and current security risks as now both the application and database are not hosted on any server).

Impact:

Server-side request forgery has a **very low** impact in my app, as this is mainly related to network security vulnerabilities, and my application is hosted on a server, for the application part of this risk, it is fairly well covered through the usage of topics such as input validation, due to this, with the current implementation of the app, there is definitely not as much which can go wrong with this risk compared to other risks.

Conclusion

I think that for an individual project for Software Engineering the security of my application is sufficient for the scope of this semester, it could surely be improved to have an improved security, particular upon some specific risks in OWASP's top 10 security risks, such as for A02 cryptographic failures, as currently the application data is insecure given that it is sent and stored as plain text (without any kind of encryption), which would be a very large security risk in a real-world application. Not all risks from OWASP's list are relevant to my application, as at least parts of several points consist of the usage of servers, which are unused by my app.

To conclude, the security of my application is mainly relative to the way in which it should be implemented from the software point-of-view opposed to the cybersecurity perspective, meaning that the backend and frontend aim to be well-designed against topics such as input validation, a proper software design and keeping software modules updated, however while it is protected against some cybersecurity principles, examples include: hashing and salting passwords and protection against SQL injection, the application is not secure against pure cybersecurity implementations, like checking passwords against a list of weak passwords, or more complex cybersecurity routines, such as the regularly performing penetration testing.